Aula 10 — Manipulação de vetores

Percorrendo listas, buscando valores, maior, menor e relatórios

Dieimes Nunes

Retomando a Aula 09

Na aula anterior, começamos a trabalhar com listas.

Vimos que listas permitem guardar vários valores em uma única variável.

Exemplo:

```
notas = [8.5, 7.0, 6.5, 9.0]
```

Hoje vamos manipular listas com mais profundidade.

Vamos usar listas para:

- pesquisar;
- contar;
- somar;
- encontrar maior e menor;
- gerar relatórios.

O problema da aula

Imagine um sistema simples de controle de notas.

Precisamos:

1. cadastrar várias notas;
2. mostrar todas as notas;
3. calcular a média;
4. identificar a maior nota;
5. identificar a menor nota;
6. contar quantas notas ficaram acima da média.

Esse tipo de problema exige percorrer a lista várias vezes.

Percorrer uma lista

Percorrer uma lista significa passar por seus elementos.

Exemplo:

```
notas = [8, 6, 9, 7]

for nota in notas:
    print(nota)
```

Esse `for` lê uma nota por vez.

A variável `nota` recebe temporariamente cada valor da lista.

Percorrer com índice

Também podemos percorrer usando índice.

```
notas = [8, 6, 9, 7]

for indice in range(len(notas)):
    print(indice, notas[indice])
```

Esse formato é útil quando precisamos:

- mostrar a posição;
- alterar o valor;
- comparar elementos próximos;
- trabalhar com duas listas relacionadas.

Atividade 1 — Mostrando notas

Crie um arquivo chamado:

`mostrar_notas.py`

O programa deve:

1. criar uma lista com cinco notas;
2. mostrar cada nota em uma linha;
3. mostrar cada nota com seu índice.

Exemplo:

```
Índice 0 - Nota 8.5
Índice 1 - Nota 7.0
```

Somando valores de uma lista

Para somar valores, usamos acumulador.

```
notas = [8, 6, 9, 7]

soma = 0

for nota in notas:
    soma = soma + nota

print(soma)
```

Resultado:

```
30
```

O acumulador guarda a soma parcial a cada repetição.

Calculando média de uma lista

Depois de somar, dividimos pela quantidade de elementos.

```
notas = [8, 6, 9, 7]

soma = 0

for nota in notas:
    soma = soma + nota

media = soma / len(notas)

print(f"Média: {media:.2f}")
```

A função `len(notas)` mostra quantas notas existem.

Atividade 2 — Média de notas

Crie um arquivo chamado:

`media_notas_vetor.py`

O programa deve:

1. pedir quantas notas serão digitadas;
2. cadastrar as notas em uma lista;
3. calcular a soma;
4. calcular a média;
5. mostrar todas as notas e a média final.

Desafio: não aceitar notas menores que 0 ou maiores que 10.

Encontrando o maior valor

Para encontrar o maior valor, podemos começar assumindo que o primeiro elemento é o maior.

```
notas = [8, 6, 9, 7]

maior = notas[0]

for nota in notas:
    if nota > maior:
        maior = nota

print(f"Maior nota: {maior}")
```

A cada repetição, o programa compara a nota atual com o maior valor encontrado até aquele momento.

Por que começar com notas[0]?

Poderíamos começar com zero?

```
maior = 0
```

Para notas, talvez funcione.

Mas para outros problemas, pode dar erro lógico.

Exemplo: lista de temperaturas negativas.

```
temperaturas = [-5, -2, -10]
```

Se começarmos com `maior = 0`, o programa diria que o maior valor é zero, mas zero nem está na lista.

Por isso, usar o primeiro elemento é mais seguro.

Atividade 3 — Maior valor

Crie um arquivo chamado:

`maior_valor.py`

O programa deve:

1. criar uma lista de valores;
2. percorrer a lista;
3. identificar o maior valor;
4. mostrar o maior valor ao final.

Depois teste com:

```
valores = [10, 5, 30, 8, 12]
```

Resultado esperado:

```
Maior valor: 30
```

Encontrando o menor valor

A lógica do menor valor é parecida.

```
notas = [8, 6, 9, 7]

menor = notas[0]

for nota in notas:
    if nota < menor:
        menor = nota

print(f"Menor nota: {menor}")
```

A diferença está no operador:

```
if nota < menor:
```

Atividade 4 — Maior e menor nota

Crie um arquivo chamado:

`maior_menor_nota.py`

O programa deve:

1. pedir quantas notas serão cadastradas;
2. guardar as notas em uma lista;
3. encontrar a maior nota;
4. encontrar a menor nota;
5. mostrar as duas no final.

Depois teste com notas diferentes e confira manualmente.

Contando valores em uma lista

Podemos contar quantos valores atendem a uma condição.

Exemplo: contar quantas notas são maiores ou iguais a 7.

```
notas = [8, 6, 9, 7, 5]

aprovados = 0

for nota in notas:
    if nota >= 7:
        aprovados = aprovados + 1

print(f"Aprovados: {aprovados}")
```

Esse é o uso de contador com lista.

Atividade 5 — Contando aprovados

Crie um arquivo chamado:

`contar_aprovados.py`

O programa deve:

1. cadastrar várias notas em uma lista;
2. contar quantas notas são maiores ou iguais a 7;
3. contar quantas notas são menores que 7;
4. mostrar o total de aprovados e reprovados.

Desafio: mostrar também a porcentagem de aprovação.

Pesquisando em uma lista

Na aula anterior, usamos `in`.

```
if nome in alunos:
    print("Encontrado")
```

Agora vamos entender como a pesquisa pode ser feita manualmente.

A ideia é percorrer a lista e comparar elemento por elemento.

Pesquisa sequencial

A **pesquisa sequencial** verifica os elementos um por um.

```
alunos = ["Ana", "Bruno", "Carla", "Diego"]

busca = input("Digite o nome: ")
encontrado = False

for aluno in alunos:
    if aluno == busca:
        encontrado = True

if encontrado:
    print("Aluno encontrado.")
else:
    print("Aluno não encontrado.")
```

Essa forma ajuda a entender como a busca funciona por dentro.

Melhorando a pesquisa

Podemos mostrar a posição do aluno encontrado.

```
alunos = ["Ana", "Bruno", "Carla", "Diego"]

busca = input("Digite o nome: ")
posicao = -1

for indice in range(len(alunos)):
    if alunos[indice] == busca:
        posicao = indice

if posicao != -1:
    print(f"Aluno encontrado no índice {posicao}")
else:
    print("Aluno não encontrado.")
```

Usamos `-1` para indicar que ainda não encontramos.

Atividade 6 — Pesquisa sequencial

Crie um arquivo chamado:

`pesquisa_sequencial.py`

O programa deve:

1. criar uma lista de produtos;
2. pedir o nome de um produto;
3. percorrer a lista manualmente;
4. informar se o produto foi encontrado;
5. se encontrado, mostrar o índice.

Não use `in` nesta atividade.

Duas listas relacionadas

Às vezes, usamos duas listas relacionadas pelo índice.

```
nomes = ["Ana", "Bruno", "Carla"]
notas = [8.5, 6.0, 9.0]
```

Relação:

```
nomes[0] → Ana     notas[0] → 8.5
nomes[1] → Bruno   notas[1] → 6.0
nomes[2] → Carla   notas[2] → 9.0
```

O índice liga uma informação à outra.

Exemplo com duas listas

```
nomes = []
notas = []

quantidade = int(input("Quantidade de alunos: "))

for contador in range(quantidade):
    nome = input("Nome: ")
    nota = float(input("Nota: "))

    nomes.append(nome)
    notas.append(nota)

for indice in range(len(nomes)):
    print(f"{nomes[indice]} - Nota: {notas[indice]}")
```

É importante manter as listas com o mesmo tamanho.

Atividade 7 — Nomes e notas

Crie um arquivo chamado:

`nomes_e_notas.py`

O programa deve:

1. cadastrar nomes em uma lista;
2. cadastrar notas em outra lista;
3. mostrar nome e nota juntos;
4. informar a situação do aluno.

Regra:

```
nota >= 7 → Aprovado
nota < 7  → Reprovado
```

Relatório final com listas

Vamos montar um relatório com:

- nomes;
- notas;
- média da turma;
- maior nota;
- menor nota;
- aprovados;
- reprovados.

Esse tipo de programa integra vários conteúdos:

- variável;
- lista;
- repetição;
- condição;
- contador;
- acumulador;
- maior e menor.

Atividade 8 — Relatório de turma

Crie um arquivo chamado:

`relatorio_turma_lista.py`

O programa deve:

1. pedir a quantidade de alunos;
2. cadastrar nome e nota de cada aluno;
3. guardar os nomes em uma lista;
4. guardar as notas em outra lista;
5. calcular a média geral;
6. identificar maior e menor nota;
7. contar aprovados e reprovados;
8. mostrar um relatório final.

Cuidados importantes

Ao trabalhar com listas, cuidado com:

- lista vazia;
- divisão por zero;
- índice fora do intervalo;
- remover elementos durante um `for`;
- listas relacionadas com tamanhos diferentes;
- esquecer de validar dados antes de adicionar.

Antes de manipular uma lista, pense:

A lista tem elementos? O índice existe? O valor está correto?

Erro com lista vazia

Veja o problema:

```
notas = []

maior = notas[0]
```

Esse código gera erro, porque não existe índice `0` em uma lista vazia.

Antes de acessar o primeiro elemento, precisamos garantir que a lista possui dados.

```
if len(notas) > 0:
    maior = notas[0]
else:
    print("Lista vazia.")
```

Atividade 9 — Tratando lista vazia

Crie um arquivo chamado:

`lista_vazia.py`

O programa deve:

1. criar uma lista vazia;
2. verificar se ela possui elementos;
3. se estiver vazia, mostrar mensagem;
4. se tiver elementos, mostrar o primeiro item.

Depois adicione valores na lista e teste novamente.

Onde estamos no curso

Concluímos uma parte importante:

- variáveis guardam um valor;
- listas guardam vários valores;
- repetições percorrem listas;
- condições analisam cada elemento;
- contadores e acumuladores geram relatórios.

Agora estamos preparados para avançar para:

- ordenação;
- pesquisa binária;
- matrizes;
- registros;
- arquivos.

Desafio final da aula

Crie um programa chamado:

`sistema_produtos.py`

O programa deve ter duas listas:

```
produtos = []
precos = []
```

Menu:

```
1 - Cadastrar produto
2 - Listar produtos
3 - Buscar produto
4 - Mostrar produto mais caro
5 - Mostrar produto mais barato
0 - Sair
```

Regras:

- produto e preço devem ficar na mesma posição das listas;
- o preço não pode ser negativo;
- a busca deve ser sequencial;
- o menu deve repetir até o usuário sair.

Fechamento

Nesta aula, manipulamos vetores usando listas em Python.

Trabalhamos com:

- percurso de listas;
- soma e média;
- maior e menor valor;
- contadores com listas;
- pesquisa sequencial;
- listas relacionadas;
- relatórios.

Na próxima etapa, vamos estudar ordenação de vetores e métodos de pesquisa com mais detalhes.